CAPSTONE PROJECT REPORT

(Project Term January-May 2026)

Real-Time Public Transport Tracking for Small Cities

Submitted by

Project Group Number: _____

Course Code: CSE439

Under the Guidance of

_____

Designation: Assistant Professor

School of Computer Science and Engineering

Lovely Professional University

Phagwara, Punjab - 144411

Name	Registration Number
Nitin Singh	_____
Yash Gupta	_____
Priyanshu Dadich	_____
Anish Mishra	
Priyanshu Kumar	_____
Gautam Kumar	_____

TOPIC APPROVAL PERFORMA

School of Computer Science and Engineering

Program: P132::B.Tech. (Computer Science and Engineering)

COURSE CODE: CSE439 REGULAR/BACKLOG: Regular GROUP NUMBER: _____

Supervisor Name: _____ UID: _____ Designation: Assistant Professor

Qualification: _____ Research Experience: _____

SR.NO.	NAME OF STUDENT	Prov. Regd. No.	BATCH	SECTION
1	Nitin Singh	_____	2022	_____
2	Yash Gupta	_____	2022	_____
3	Priyanshu Dadich	_____	2022	_____
4	Anish Mishra	_____	2022	_____
5	Priyanshu Kumar	_____	2022	_____
6	Gautam Kumar	_____	2022	_____

SPECIALIZATION AREA: Full Stack Development, Cloud-Enabled Mobility Systems

PROPOSED TOPIC: Real-Time Public Transport Tracking for Small Cities

Supervisor Signature: _____

Qualitative Assessment of Proposed Topic by PAC

Sr.No.	Parameter	Rating (out of 10)
1	Project Novelty	_____
2	Project Feasibility	_____
3	Project Academic Inputs	_____
4	Project Supervision	_____
5	Social Applicability	_____
6	Future Scope	_____

PAC Committee Members: _____

Final Topic Approved by PAC: Real-Time Public Transport Tracking for Small Cities

Overall Remarks: _____

DECLARATION

We hereby declare that the project work entitled **"Real-Time Public Transport Tracking for Small Cities"** is an authentic record of our own work carried out as requirements of Capstone Project for the award of B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara, under the guidance of _____ during January to May 2026.

All information furnished in this capstone project report is based on our own implementation, testing, and analysis and is genuine.

Project Group Number: _____

Name of Student 1: Nitin Singh

Registration Number: _____

Name of Student 2: Yash Gupta

Registration Number: _____

Name of Student 3: Priyanshu Dadich

Registration Number: _____

Name of Student 4: Anish Mishra

Registration Number: _____

Name of Student 5: Priyanshu Kumar

Registration Number: _____

Name of Student 6: Gautam Kumar

Registration Number: _____

Signatures:

Nitin Singh: _____ Date: _____

Yash Gupta: _____ Date: _____

Priyanshu Dadich: _____ Date: _____

Anish Mishra: _____ Date: _____

Priyanshu Kumar: _____ Date: _____

Gautam Kumar: _____ Date: _____

CERTIFICATE

This is to certify that the declaration statement made by this group of students is correct to the best of my knowledge and belief. They have completed this Capstone Project under my guidance and supervision. The present work is the result of their original investigation, effort, and study.

No part of this work has ever been submitted for any other degree at any University. The project is fit for submission and partial fulfillment of the conditions for the award of B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara.

Signature and Name of the Mentor: _____

Designation: Assistant Professor

School of Computer Science and Engineering, Lovely Professional University, Phagwara

Date: _____

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to all who contributed to the successful completion of this capstone project. We are deeply thankful to our mentor for constant guidance, technical support, and continuous motivation throughout the project lifecycle.

We also extend gratitude to the faculty members and administration of the School of Computer Science and Engineering for providing the required infrastructure and academic environment.

Finally, we thank our team members for coordinated work and our families and friends for their support.

TABLE OF CONTENTS

1. INTRODUCTION

1.1. Background of the Study

1.2. About the Project

1.3. Objectives

1.4. Scope

2. PROFILE OF THE PROBLEM, RATIONALE AND SCOPE OF THE STUDY

2.1. Problem Statement

2.2. Key Problems Identified

2.3. Rationale for the Study

2.4. Scope of the Study

3. EXISTING SYSTEM

3.1. Introduction

3.2. Existing Software

3.3. DFD for Present System

3.4. What Is New in the System to Be Developed

4. PROBLEM ANALYSIS

4.1. Product Definition

4.2. Feasibility Analysis

4.2.1. Technical feasibility

4.2.2. Operational feasibility

4.2.3. Economic feasibility

4.3. Project Plan

5. SOFTWARE REQUIREMENT ANALYSIS

5.1. Introduction

5.2. General Description

5.2.1. Product perspective

5.2.2. Product functions

5.2.3. User characteristics

5.3. Specific Requirements

5.3.1. Functional requirements

5.3.2. Non-functional requirements

5.3.3. Hardware requirements

5.3.4. Software requirements

6. DESIGN

6.1. System Design

6.2. Design Notations

6.2.1. Database schema — bus model

6.2.2. Database schema — trip model

6.2.3. Database schema — location model

6.2.4. Entity-relationship diagram

6.3. Detailed Design

6.3.1. Backend module structure

6.3.2. Frontend/mobile component structure

6.3.3. Trip lifecycle logic

6.3.4. Real-time location update logic

6.3.5. Public tracking workflow

6.4. Flowcharts

6.4.1. Driver trip start flowchart

6.4.2. Location transmission flowchart

6.4.3. Public bus tracking flowchart

6.5. Pseudo Code

6.5.1. Driver start trip logic

6.5.2. Location update backend logic

7. TESTING

7.1. Functional Testing

7.2. Structural Testing

7.3. Levels of Testing

7.3.1. Unit testing

7.3.2. Integration testing

7.3.3. System testing

7.4. Testing the Project

8. IMPLEMENTATION

8.1. Implementation of the Project

8.1.1. Backend implementation

8.1.2. Frontend/mobile implementation

8.2. Conversion Plan

8.3. Post-Implementation and Software Maintenance

9. PROJECT LEGACY

9.1. Current Status of the Project

9.2. Remaining Areas of Concern

9.3. Technical and Managerial Lessons Learnt

9.3.1. Technical lessons

9.3.2. Managerial lessons

10. USER MANUAL

10.1. System Setup and Installation

10.1.1. Prerequisites

10.1.2. Backend setup steps

10.1.3. User web app setup steps

10.1.4. Driver app setup steps

10.2. Commuter User Guide

10.2.1. Viewing active buses by route

10.2.2. Monitoring bus status and live position

10.3. Driver User Guide

10.3.1. Logging in as a driver

10.3.2. Starting a trip

10.3.3. Sending and verifying location updates

10.3.4. Reporting a breakdown

10.3.5. Ending a trip

11. SOURCE CODE AND SYSTEM SNAPSHOTS

11.1. GitHub Repository Link

11.2. Key Source Code Excerpts

11.2.1. Socket.IO server initialization — server.js

11.2.2. Role-based authentication middleware — auth.js

11.2.3. Trip start route — trip.js

11.2.4. Location ingestion route — location.js

11.2.5. Public active buses endpoint — public.js

11.2.6. Driver location background task — location.js

11.3. System Snapshots

11.3.1. Driver login screen

11.3.2. Driver active trip screen

11.3.3. User live bus map screen

11.3.4. Admin fleet management screen

12. BIBLIOGRAPHY

1. INTRODUCTION

1.1. Background of the Study

Urban mobility in small cities is highly dependent on buses but usually lacks live digital transport information systems. Commuters face uncertainty about bus arrival times and service availability due to manual operations and static timetables.

1.2. About the Project

The project implements a real-time public transport tracking platform with a role-based backend API, a driver mobile app for GPS updates, a user-facing map app for bus tracking, and an admin panel for operational management. The system architecture combines mobile GPS, cloud APIs, WebSocket streaming, and map visualization.

1.3. Objectives

4. Provide route-wise real-time bus visibility for commuters.
5. Digitize fleet operations for administrators and drivers.
6. Enable secure authentication and role-specific workflows.

4. Maintain trip and location history for operational analysis.

5. Support low-cost, scalable deployment for small-city use cases.

1.4. Scope

The scope includes live bus tracking, route filtering, trip lifecycle management, and driver incident reporting. ETA prediction with advanced traffic modeling, e-ticketing, and payment integration are outside current scope.

2. PROFILE OF THE PROBLEM, RATIONALE AND SCOPE OF THE STUDY

2.1. Problem Statement

Commuters in small cities often lack accurate real-time information regarding bus position, delays, and service status. Authorities also struggle with fleet visibility and decision-making due to fragmented operational data.

2.2. Key Problems Identified

- No public real-time bus visibility.
- No structured trip-state lifecycle.
- No centralized location history for operations.
- Delayed incident reporting and actionability.
- Dependence on manual communication channels.

2.3. Rationale for the Study

By integrating mobile GPS, cloud APIs, and map interfaces, the system reduces passenger uncertainty and improves operational transparency. The technology stack is practical for small cities due to its low cost and modular deployability.

2.4. Scope of the Study

The study covers software analysis, design, implementation, and validation of the transport tracking platform across backend and client applications.

3. EXISTING SYSTEM

3.1. Introduction

Existing transport management in small cities typically uses manual logs, phone calls, and non-digital public communication methods.

3.2. Existing Software

- Manual depot-level route boards.
- Spreadsheet-based internal records.
- Informal phone-based driver coordination.

3.3. DFD for Present System

Figure 1: DFD for Present Manual Public Transport Monitoring System.

(Insert figure in final DOC/PDF.)

3.4. What Is New in the System to Be Developed

- Live GPS location updates from drivers.
- Bus-level WebSocket broadcasts to clients.
- Public route listing and active bus APIs.
- Admin bus/driver management and assignment.
- Incident (breakdown) reporting in trip lifecycle.

4. PROBLEM ANALYSIS

4.1. Product Definition

The product is a full-stack public transport tracking system with multiple clients and a centralized backend. It tracks buses in real time, exposes live route feeds, and supports operational workflows for drivers and admins.

4.2. Feasibility Analysis

4.2.1. Technical feasibility

The system uses open-source technologies: Node.js, Express, MongoDB, Socket.IO, React, React Native (Expo), and map SDKs. These are mature and widely documented.

4.2.2. Operational feasibility

The user flow is straightforward: admins assign buses, drivers start/end trips and push location, users observe buses on map by route.

4.2.3. Economic feasibility

No licensing cost for core stack. Cloud hosting and DB usage remain economical for pilot deployments.

4.3. Project Plan

Phase	Activities	Duration
Phase 1	Requirement analysis and architecture planning	Week 1-2
Phase 2	Data model and API design	Week 3-4
Phase 3	Backend API + Socket.IO implementation	Week 5-8
Phase 4	Driver and user app integration	Week 9-13
Phase 5	Testing and bug fixing	Week 14-16
Phase 6	Documentation and final report	Week 17-18

5. SOFTWARE REQUIREMENT ANALYSIS

5.1. Introduction

This chapter describes requirements derived from the transport tracking problem and validated through implementation.

5.2. General Description

5.2.1. Product perspective

The platform is a layered client-server system:

- Driver app -> backend location/trip APIs
- Backend -> MongoDB + WebSocket events
- User app -> public APIs + live sockets
- Admin panel -> protected APIs

5.2.2. Product functions

- Admin/driver/user authentication
- Bus and driver CRUD + assignment
- Trip start/end/breakdown
- Location ingestion and history
- Public active bus feed and route feed
- Live map update via sockets

5.2.3. User characteristics

- Commuters: non-technical end users, map-based interaction.
- Drivers: mobile-first operational workflow.
- Admins: web-based fleet and operations control.

5.3. Specific Requirements

5.3.1. Functional requirements

7. System shall provide role-based login for admin, driver, and user.
8. Driver shall start trip only when assigned a bus.

9. System shall create and maintain one active trip per driver at a time.

4. Driver app shall transmit GPS coordinates periodically.

5. Backend shall validate trip ownership for location updates.

6. Backend shall store location with speed, heading, and timestamp.

7. Backend shall update current bus location on each valid location message.

8. Backend shall emit `location-update` to subscribed clients.

9. Public endpoints shall provide active buses and available routes.

10. Driver shall report breakdown reason during active trip.

11. Driver shall end trip with end location capture.

12. Admin shall view and manage buses/drivers/trips.

5.3.2. Non-functional requirements

10. Security: JWT protection on restricted routes.
11. Performance: low-latency location update and broadcasting.
12. Reliability: graceful handling of intermittent network conditions.

4. Scalability: room-based socket subscriptions by bus.

5. Maintainability: modular route-model-controller organization.

6. Portability: web + mobile clients.

5.3.3. Hardware requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB minimum; 8 GB recommended
Storage	10 GB free disk space
Network	Stable internet for real-time sync
Client Device	Smartphone (driver), web browser device (user/admin)

5.3.4. Software requirements

Software / Library	Version	Purpose

Node.js	v18.x or higher	Backend runtime
npm	v9.x or higher	Package manager
Express.js	^4.18.2	REST API framework
MongoDB Atlas / MongoDB	v6.x+	Database
Mongoose	^8.0.3	ODM
jsonwebtoken	^9.0.2	JWT auth
bcryptjs	^2.4.3	Password hashing
Socket.IO	^4.6.1	Real-time events
React	v18.x	Web apps
Vite	v5.x	Web build tooling
Expo + React Native	SDK 49 / RN 0.72	Driver mobile app
Leaflet / React-Leaflet	Latest compatible	Live map rendering

6. DESIGN

6.1. System Design

The architecture has four layers:

13. Vehicle Tracking Layer (driver device GPS)
14. Communication Layer (HTTP + WebSocket)
15. Cloud Processing Layer (Express + MongoDB + Socket.IO)

4. Application Layer (admin, driver, commuter interfaces)

Figure 2: System Architecture Diagram of Real-Time Public Transport Tracking System.

(Insert figure in final DOC/PDF.)

6.2. Design Notations

6.2.1. Database schema - bus model

Field	Type / Constraints	Description
busNumber	String, unique, required	Operational bus number
plateNumber	String, unique, required	Vehicle registration number
capacity	Number, min 1	Passenger capacity
route	String, required	Assigned route name
routeCoordinates	Array(lat,lng)	Route polyline coordinates
assignedDriver	ObjectId(Driver)	Linked driver
status	Enum(active,inactive,maintenance,breakdown)	Bus state
currentLocation	{lat,lng,timestamp}	Last known bus position

6.2.2. Database schema - trip model

Field	Type / Constraints	Description
bus	ObjectId(Bus), required	Trip bus reference

driver	ObjectId(Driver), required	Trip driver reference
startTime	Date, required	Trip start timestamp
endTime	Date	Trip end timestamp
status	Enum(active,completed,cancelled)	Trip lifecycle state
startLocation	{lat,lng}	Trip start coordinates
endLocation	{lat,lng}	Trip end coordinates
breakdownReported	Boolean	Incident flag
breakdownReason	String	Incident details

6.2.3. Database schema - location model

Field	Type	Description
trip	ObjectId(Trip)	Trip reference
bus	ObjectId(Bus)	Bus reference
driver	ObjectId(Driver)	Driver reference
coordinates	{lat,lng}	Captured GPS point
speed	Number	Current speed
heading	Number	Direction heading
timestamp	Date	Capture time

6.2.4. Entity-relationship diagram

Figure 3: ER Diagram for Bus-Trip-Location-Driver-Admin-User Entities.

(Insert figure in final DOC/PDF.)

6.3. Detailed Design

6.3.1. Backend module structure

- `server.js`: Express + Socket.IO bootstrap and route mounting.
- `models/`: `Bus`, `Trip`, `Location`, `Driver`, `Admin`, `User`.
- `routes/`: auth, buses, drivers, trips, locations, public.
- `middleware/auth.js`: role-based JWT verification.

6.3.2. Frontend/mobile component structure

- `user-app`: route filter + map + live bus status cards.
- `driver-app`: login, start trip, end trip, breakdown, periodic update loop.
- `admin-panel`: operational management workflows.

6.3.3. Trip lifecycle logic

Driver starts trip -> bus status becomes active -> trip status active -> location events stream -> driver ends trip -> status completed and bus inactive.

6.3.4. Real-time location update logic

Driver app sends payload every ~5 seconds -> backend validates active trip and ownership -> location stored -> bus current location updated -> event emitted to bus room.

6.3.5. Public tracking workflow

User app fetches routes and active buses from public endpoints, subscribes to socket updates, and renders map marker movement and route polyline.

6.4. Flowcharts

6.4.1. Driver trip start flowchart

Figure 4: Driver Trip Start Flowchart.

(Insert figure)

6.4.2. Location transmission flowchart

Figure 5: GPS Update Transmission and Processing Flowchart.

(Insert figure)

6.4.3. Public bus tracking flowchart

Figure 6: Commuter Live Tracking Flowchart.

(Insert figure)

6.5. Pseudo Code

6.5.1. Driver start trip logic

FUNCTION startTrip(driver, startLocation):

IF driver.assignedBus is null -> RETURN error

IF active trip already exists -> RETURN error

SET bus.status = active

SET bus.currentLocation = startLocation

CREATE trip(status=active, startTime=now)

EMIT trip-started and location-update

RETURN trip

6.5.2. Location update backend logic

FUNCTION updateLocation(driver, tripId, coordinates, speed, heading):

VALIDATE request body

trip = FIND trip by tripId

IF trip not found OR trip.driver != driver OR trip.status != active -> RETURN error

INSERT location record

UPDATE bus.currentLocation

EMIT location-update to subscribed clients

RETURN success

7. TESTING

7.1. Functional Testing

TC No.	Test Description	Expected Result	Status
TC-01	Driver login with valid credentials	200 with JWT and driver profile	Pass
TC-02	Driver login with wrong password	401 invalid credentials	Pass
TC-03	Start trip without assigned bus	400 no bus assigned	Pass
TC-04	Start trip with assigned bus	201 trip created active	Pass
TC-05	Location update on active trip	201 location saved and emitted	Pass
TC-06	Location update on inactive trip	400 trip is not active	Pass
TC-07	Breakdown report	200 breakdown logged	Pass
TC-08	End trip with end location	200 trip completed	Pass
TC-09	Public active buses endpoint	200 route-wise active list	Pass
TC-10	Public routes endpoint	200 distinct sorted routes	Pass

7.2. Structural Testing

Authentication checks, ownership validation, and trip-state branch conditions were verified in route handlers.

7.3. Levels of Testing

7.3.1. Unit testing

Validation rules and schema field constraints checked at module level.

7.3.2. Integration testing

Driver app -> backend -> MongoDB -> socket -> user app map flow tested end to end.

7.3.3. System testing

Complete deployment tested with parallel clients (admin, driver, commuter) and real-time event propagation.

7.4. Testing the Project

The implemented core features were tested in development conditions and showed stable real-time behavior across primary workflows.

8. IMPLEMENTATION

8.1. Implementation of the Project

8.1.1. Backend implementation

The backend uses modular API groups and Socket.IO server integration with MongoDB models for buses, trips, and locations.

8.1.2. Frontend/mobile implementation

The commuter app visualizes live map markers and route filters. The driver app captures and transmits location periodically and supports background location tasks.

8.2. Conversion Plan

16. Configure admin, bus records, and driver assignment.
17. Pilot selected routes with trained drivers.
18. Enable public route view for commuters.

4. Expand after pilot performance review.

8.3. Post-Implementation and Software Maintenance

- periodic dependency audit and upgrades
- DB index optimization for location queries
- improved monitoring and alerting for stale updates
- future ETA prediction enhancement

9. PROJECT LEGACY

9.1. Current Status of the Project

Core platform features (authentication, trip lifecycle, location streaming, and public tracking) are implemented and integrated.

9.2. Remaining Areas of Concern

- Advanced ETA model not yet integrated.
- Production-grade alerting and analytics dashboards pending.
- Broader field validation needed under varied connectivity.

9.3. Technical and Managerial Lessons Learnt

9.3.1. Technical lessons

- Event-driven real-time architecture improves tracking responsiveness.
- Strict route-level authorization is critical for data integrity.
- Driver ownership checks prevent unauthorized location writes.

9.3.2. Managerial lessons

- API contract finalization before UI implementation reduces rework.
- Pilot rollout by route helps controlled deployment.
- Continuous testing is essential in real-time systems.

10. USER MANUAL

10.1. System Setup and Installation

10.1.1. Prerequisites

- Node.js v18+
- MongoDB URI / Atlas cluster
- Expo setup for driver app
- Modern web browser

10.1.2. Backend setup steps

19. `cd backend`
20. `npm install`
21. create `.env` with `MONGODB_URI`, `JWT_SECRET`, `PORT`

4. `npm run dev`

10.1.3. User web app setup steps

22. `cd user-app`
23. `npm install`
24. `npm run dev`

10.1.4. Driver app setup steps

25. `cd driver-app`
26. `npm install`
27. `npx expo start` (or EAS build for APK)

10.2. Commuter User Guide

10.2.1. Viewing active buses by route

Open user app, choose route filter, and view active buses in list and map view.

10.2.2. Monitoring bus status and live position

Observe marker updates in real-time and check status labels (active/breakdown).

10.3. Driver User Guide

10.3.1. Logging in as a driver

Use authorized driver credentials on the driver app login screen.

10.3.2. Starting a trip

Tap "Start Trip" once assigned bus is available.

10.3.3. Sending and verifying location updates

Keep location permissions enabled; app sends updates periodically.

10.3.4. Reporting a breakdown

Use "Report Breakdown" and enter incident reason.

10.3.5. Ending a trip

Tap "End Trip" at route completion to close active session.

11. SOURCE CODE AND SYSTEM SNAPSHOTS

11.1. GitHub Repository Link

Project code repository link: _____

11.2. Key Source Code Excerpts

11.2.1. Socket.IO server initialization - `backend/server.js`

11.2.2. Role-based authentication middleware - `backend/middleware/auth.js`

11.2.3. Trip start route - `backend/routes/trip.js`

11.2.4. Location ingestion route - `backend/routes/location.js`

11.2.5. Public active buses endpoint - `backend/routes/public.js`

11.2.6. Driver location background task - `driver-app/utils/location.js`

11.3. System Snapshots

11.3.1. Driver login screen

11.3.2. Driver active trip screen

11.3.3. User live bus map screen

11.3.4. Admin fleet management screen

(Insert actual screenshots in final DOC/PDF)

12. BIBLIOGRAPHY

28. Zhou, P., Zheng, Y., Li, M. (2012). *How Long to Wait? Predicting Bus Arrival Time with Mobile Phone Based Participatory Sensing.*
29. Biagioni, J. et al. (2011). *EasyTracker: Automatic Transit Tracking, Mapping, and Arrival Time Prediction Using Smartphones.*
30. Hu, H., Liu, Y., Wang, J. (2011). *Real-Time Bus Arrival Time Prediction Using GPS Data.*

4. Express.js Documentation. <https://expressjs.com/>

5. Mongoose Documentation. <https://mongoosejs.com/docs/>

6. Socket.IO Documentation. <https://socket.io/docs/>

7. React Documentation. <https://react.dev/>

8. Expo Documentation. <https://docs.expo.dev/>

9. Leaflet Documentation. <https://leafletjs.com/>

10. MongoDB Atlas Documentation. <https://www.mongodb.com/docs/atlas/>